

Bikini Bottom Artificial Life: A Multi-Agent Simulation with Perception-Driven Behavior, Internal Hunger State, and Environmental Hazards

Leica Shen
Computer Science Department
University of California, Los Angeles
Email: jws@ucla.edu

Yiran Wu
Computer Science Department
University of California, Los Angeles
Email: wuyiran12@ucla.edu

Zhengli Shang
Computer Science Department
University of California, Los Angeles
Email: z4shang@ucla.edu

Zijie Zhang
Computer Science Department
University of California, Los Angeles
Email: zijiezh@ucla.edu

Abstract—We present a browser-based three-dimensional artificial-life simulation inspired by the animated setting of *Bikini Bottom*. Fish agents move through a stylized underwater volume using a local perception-action loop that combines classical Reynolds-style steering forces with two artificial-life-oriented extensions: an internal hunger state that gates food-seeking motivation, and school-level target modes that produce stable explore, food-loiter, or food-avoidance behavior at the group scale. A procedurally animated jellyfish swarm acts as a periodic bait/hazard, and static landmark footprints impose collision constraints. The system is implemented in TypeScript with React, Vite, Three.js, and React Three Fiber, runs at interactive frame rates in a standard web browser, and exposes the main behavior weights through an interactive controls panel so that emergent group dynamics can be inspected and tuned in real time.

I. INTRODUCTION

Artificial Life seeks to capture lifelike behavior from simple local rules rather than from centralized control. Since Langton coined the term in 1986 and Reynolds’ boids model [1] showed that realistic flocking can emerge from three local steering rules — alignment, cohesion, and separation — a central question of the field has been how much expressive group behavior can be recovered from perception-action loops that use only local information. Braitenberg’s synthetic vehicles [2] make the same point from the opposite direction: behaviors that look intentional from the outside can often be explained by very simple wirings between sensors and actuators. Tu’s artificial fish [3] brought these ideas into a continuous 3D underwater setting and added a layer of internal state — hunger, fear, libido — on top of the local perception, demonstrating that a small amount of motivational state is enough to produce visibly purposeful swimming, feeding, and predator-avoidance behavior.

The specific question we wanted to ask in this project is whether the classical local-perception philosophy still carries

an entire *scene* — not just an isolated flock — once that scene contains multiple coexisting species, a shared food source, a moving hazard, and static landmark geometry. A naive composition of Reynolds-style steering with a single food attractor turns out to produce a visually unsatisfying answer: every school eventually collapses onto the food and the four species merge into one mixed cluster, so the group dynamics that motivated the project disappear the moment feeding begins. Our extensions are aimed directly at that failure mode. We keep the local-perception backbone of boids, but we gate food desire by a per-agent hunger state in the spirit of Tu’s artificial fish, and we lift wander-target selection from the individual to the school so that group-scale exploration, food-loitering, and post-feeding drift remain stable behaviors rather than per-frame noise. The aim is not to advance the state of the art in flocking but to test whether these two small additions are enough to keep a multi-species, hazard-aware, browser scale Alife scene legible and tunable in real time.

Our scene is inspired directly by the world of *SpongeBob SquarePants*. *Bikini Bottom* is a particularly inviting target for an Alife project because its visual language already encodes most of the ingredients we care about: the Krabby Patty acts as a famous, instantly recognizable food attractor; the jellyfish fields play the role of roaming environmental hazards; and the surrounding reef is populated by a diverse cast of small fish in distinct shapes and colors. The animated source material does much of our exposition for us — a viewer already understands that fish should crowd toward food, that jellyfish should be feared and avoided, and that different species should look and move differently — which lets us focus the simulation on *whether* simple local rules can reproduce those behaviors rather than on *explaining* the scene. We model four visually distinct fish species (reef, blue, puffer, and long fish), three ground-walking character agents (*SpongeBob*, *Patrick*, and

Squidward) that share the same steering scaffold but stay locked to the seafloor, a Krabby Patty food source, and a small jellyfish swarm that periodically migrates toward the bait area. The resulting scene is recognizable as Bikini Bottom while remaining grounded in a single perception–action loop that we can analyze and tune.

II. RELATED WORK

A. Boids and Flocking

Reynolds’ boids model [1] demonstrated that realistic flocking can emerge from three local rules: alignment, cohesion, and separation. Our system uses the same three rules as a backbone but combines them with explicit food attraction, hazard avoidance, boundary steering, and a school-level wander target.

B. Artificial Fish with Internal State

Tu’s artificial fish [3] introduced perception, locomotion, and behavior controllers in which internal state variables (such as hunger and fear) directly modulate the active behavior. Our work follows the same spirit but stays deliberately simpler: a single hunger scalar per fish, combined with a small number of school-level target modes, is enough to recover legible feeding cycles.

C. Synthetic Behavior from Simple Sensors

Braitenberg’s synthetic vehicles [2] argue that complex-looking behaviors can be interpreted in terms of very simple sensor-to-actuator wiring. The Alife framing we use — in which each visible movement should be explainable from local perception, hunger, and school state — is directly motivated by that point of view.

III. SYSTEM OVERVIEW

We organize the simulation as three communicating layers: a *perception* layer that gathers local neighborhood information, a *behavior* layer that computes weighted steering forces, and a *motion* layer that integrates velocity and position under boundary and obstacle constraints. Two cross-cutting state variables — a per-agent internal hunger and a per-school target mode — modulate the behavior layer. Figure 1 summarizes the data flow.

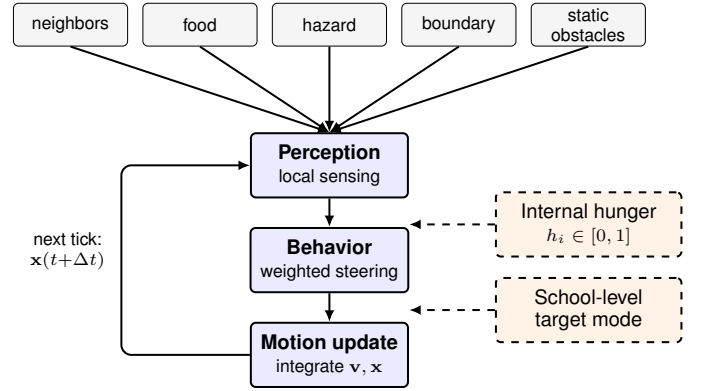


Fig. 1. High-level block diagram of the simulation. Each fish agent runs an independent perception–action loop (solid arrows); the internal hunger scalar and the school-level target mode modulate the behavior block (dashed arrows). The only cross-agent coupling is through (a) the school-level target mode shared by members of the same species, and (b) the implicit coupling created by the local neighborhood query.

A. Simulation Tick

At each animation frame the simulation advances by a clamped time step Δt (capped at 40 ms to keep the integrator stable when the browser stalls). Every fish agent then runs the same loop:

- 1) Sense neighbors within `perceptionRadius`.
- 2) Sense food and hazard within their respective alert ranges.
- 3) Compute the weighted sum of steering forces.
- 4) Clamp the force by `maxForce`, integrate velocity, and clamp velocity by `maxSpeed`.
- 5) Integrate position and resolve boundary and static-obstacle constraints.
- 6) Update internal hunger based on proximity to food.

Ground-walking character agents (SpongeBob, Patrick, Squidward) share the same loop but with their vertical motion locked to the seafloor.

B. Rendering Layer

The rendering layer is a Three.js scene driven by React Three Fiber. Imported `.glb` models [4]–[6] are cloned per agent through `SkeletonUtils` so that each instance can carry its own animation state, while the jellyfish swarm is animated procedurally in JavaScript rather than from a baked clip.

IV. AGENT MODEL AND PERCEPTION

A. Agent State

Every agent is described by the tuple $\langle \mathbf{x}, \mathbf{v}, s, g, h, b \rangle$, where

- $\mathbf{x} \in \mathbb{R}^3$ is position,
- $\mathbf{v} \in \mathbb{R}^3$ is velocity,
- s is the species (*reef*, *blue*, *puffer*, *long*, plus three character types),
- g is the group identifier,
- $h \in [0, 1]$ is the internal hunger scalar (§VI),
- $b \in \{0, 1\}$ indicates whether the agent is ground-locked.

TABLE I
DEFAULT BEHAVIOR PARAMETERS (SRC/SIM/SIMULATION.TS).

Parameter	Value	Purpose
Agent count	48	Total simulated agents
Perception radius	4.2	Local sensing range
Food attraction range	$r_p + 1.4$	Fish food alert range
Initial hunger range	0.62–1.0	Starting hunger
Hunger drop near food	0.16/s	Feeding rate
Hunger recovery away	0.034/s	Recovery rate
Hungry status threshold	0.6	Red status dot
Full food desire floor	0.5	Residual pull when full
Food-loiter weight threshold	2.0	Switches school mode
School wander interval	8–18 s	Re-pick exploration target
Jellyfish bait interval	30–45 s	Periodic bait visit
Cross-species sep. mult.	2.55	Larger range to other species

There is no global blackboard. The only inter-agent coupling is through the perception query and through the school-level target mode shared by agents with the same g .

B. Perception

Each agent senses neighbors using a simple radius query against the global agent list:

$$\mathcal{N}(i) = \{j : \|\mathbf{x}_j - \mathbf{x}_i\| \leq r_p, j \neq i\}.$$

The food attractor and the jellyfish hazard use their own alert ranges derived from r_p . Notably, the food alert range is $\text{perceptionRadius} + 1.4$ for fish but is expanded to the full scene diagonal for ground-walking characters, because the character agents have no school-level wander force and would otherwise never discover the Krabby Patty.

C. Default Parameters

Table I lists the default behavior parameters used in the rest of the report.

V. STEERING BEHAVIORS

At each tick, agent i computes a weighted sum of steering vectors:

$$\begin{aligned} \mathbf{F}_i = & w_{\text{food}} d_i \mathbf{F}_i^{\text{food}} + w_{\text{haz}} \mathbf{F}_i^{\text{haz}} + w_{\text{sep}} \mathbf{F}_i^{\text{sep}} \\ & + w_{\text{al}} \mathbf{F}_i^{\text{al}} + w_{\text{co}} \mathbf{F}_i^{\text{co}} + w_{\text{b}} \mathbf{F}_i^{\text{b}} + w_{\text{ob}} \mathbf{F}_i^{\text{ob}} + w_{\text{wa}} \mathbf{F}_i^{\text{wa}}, \end{aligned} \quad (1)$$

where $d_i \in [0.5, 1]$ is the hunger-modulated food desire defined in Equation 4. The force is clamped by maxForce , integrated into velocity, and the velocity is clamped by maxSpeed :

$$\mathbf{v}_i(t+\Delta t) = \text{clamp}(\mathbf{v}_i(t) + \mathbf{F}_i \Delta t, v_{\text{max}}), \quad (2)$$

$$\mathbf{x}_i(t+\Delta t) = \mathbf{x}_i(t) + \mathbf{v}_i(t+\Delta t) \Delta t. \quad (3)$$

A. Food Attraction

Each agent inside the food alert range receives a desired velocity toward the Krabby Patty whose magnitude decays linearly with distance. A constant baseline term keeps the pull from vanishing at the edge of the alert range, and ground-walking characters use a larger baseline so that they can be drawn from anywhere in the scene. Agents that are inside the collision radius are explicitly pushed back out. Figure 2 shows a typical snapshot of fish converging on the Krabby Patty.

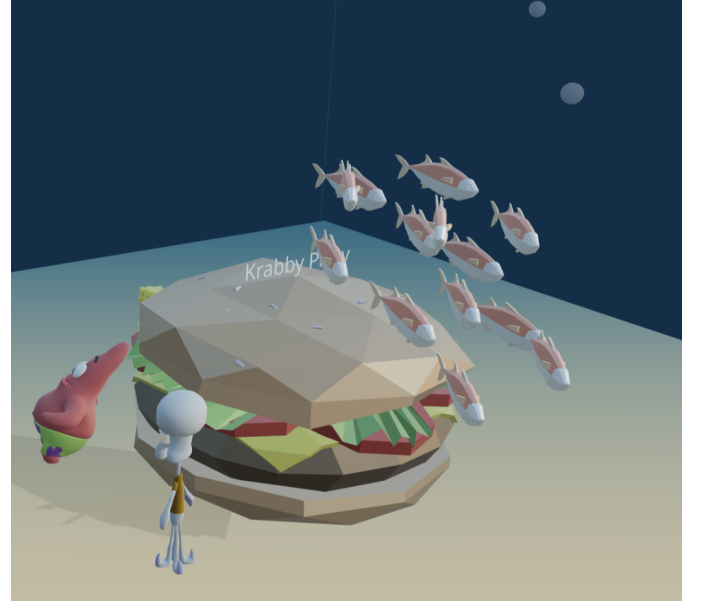


Fig. 2. Food attraction in action: fish from several schools steer toward the Krabby Patty once it enters their alert range. Hunger-driven desire (d_i) modulates the magnitude of this pull per agent.

B. Hazard Avoidance

Agents inside the union of the hazard radius and the perception radius compute a repulsion vector whose magnitude grows with the inverse alert-distance ratio. The same swarm position is used by all fish, so hazard avoidance produces visible coordinated flight bursts when the jellyfish enter a school's neighborhood.

C. Separation, Alignment, Cohesion

Separation, alignment, and cohesion follow Reynolds-style averaging over $\mathcal{N}(i)$. Two changes are worth highlighting:

- 1) **Cross-species separation.** The separation radius is widened by $2.55\times$ when the neighbor is of a different species, and the desired speed is scaled by $1.35\times$, so different schools repel more strongly than they mix.
- 2) **Same-species alignment/cohesion only.** Alignment and cohesion only average over neighbors with the same g . This is what prevents schools from collapsing into one mixed cluster after they all visit the food source.

Figure 3 shows the resulting group dynamics: each species forms a recognizable school, and the four schools remain visually distinct even when they share the simulation volume.

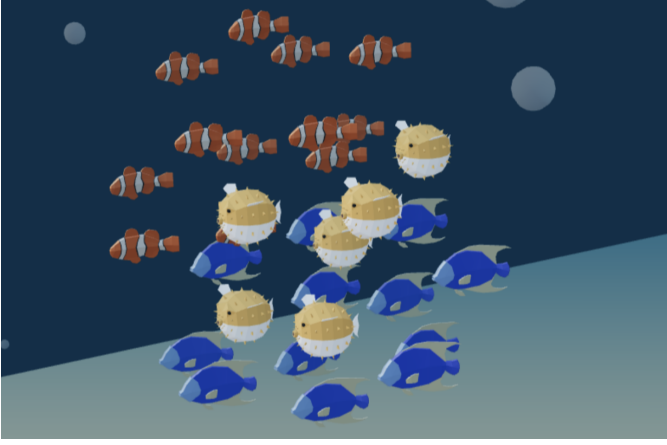


Fig. 3. Emergent group behavior. Same-species alignment and cohesion plus widened cross-species separation produce four visually distinct schools that coexist without merging into a single mixed cluster.

D. Wander Toward a School-Level Target

Rather than letting every fish pick its own wander target, we keep the wander target at the school level (see §VI-B). Each fish then simply steers toward its school’s current wanderTarget.

E. Boundary Steering

A margin-based force pushes agents back into the simulation volume whenever they enter the soft boundary band. This is a smoothed version of axis-aligned reflection and avoids the visible “slamming” that a hard boundary would produce.

F. Proactive Obstacle Avoidance

To avoid head-on collisions with static landmarks, each agent projects a horizontal lookahead along its velocity and applies a *lateral* (perpendicular) push around any cylindrical obstacle in its predicted path. The lookahead grows with speed and the push magnitude grows with both how close ahead and how close to the path centerline the obstacle is. This produces a visible “slipping past” motion rather than a stall against the obstacle. After velocity integration, a final `keepOutOfGroundObstacle` sweep guarantees no penetration in the rare case where the proactive steer was insufficient.

VI. INTERNAL HUNGER AND SCHOOL-LEVEL TARGET MODES

A naive boids-plus-attractor implementation produces a single visible behavior: every school converges onto the food source and stays there. The two extensions described here are what make the final scene legible: each fish’s motivation is observable, and full schools can either loiter or drift away in a controlled fashion.

A. Per-Agent Hunger

Each agent carries a hunger scalar $h \in [0, 1]$ where 1 means very hungry and 0 means full. The update rule is

$$h(t+\Delta t) = \text{clamp}\left(h(t) + \begin{cases} -\rho_{\text{feed}} \Delta t & \text{if near food,} \\ +\rho_{\text{recov}} \Delta t & \text{otherwise,} \end{cases} 0, 1\right),$$

with $\rho_{\text{feed}} = 0.16/\text{s}$ and $\rho_{\text{recov}} = 0.034/\text{s}$. The food force is multiplied by a hunger-modulated desire

$$d_i = 0.5 + 0.5 h_i, \quad d_i \in [0.5, 1.0], \quad (4)$$

so that a fully fed fish still feels half of the configured food attraction while a starving fish feels all of it. This is what makes the Food-attraction slider visibly different before and after feeding without ever fully turning the pull off.

B. School-Level Target Modes

Wander target selection is tracked per school rather than per fish so that wandering is stable at the group scale. Each school’s behavior state holds a `wanderTarget`, a re-pick time, and one of three target modes:

- **explore.** Normal wandering through the simulation volume.
- **avoid-food.** Triggered when the entire school is full *and* the food attraction weight is below the food-loiter threshold. The school re-picks a target away from the Krabby Patty so it can drift off and feeding cycles can restart.
- **food-loiter.** Triggered when the entire school is full *and* the food attraction weight is above the threshold. The school picks a target inside the feeding ring (but outside the collision radius), producing visible “hanging around the food” behavior.

The explicit mode is what prevents target thrashing. Without it, a full school under high food attraction would pick a fresh random loiter target every frame and never settle.

C. Cross-Species Separation Revisited

The hunger-driven feeding cycle would otherwise let every species converge onto the food at the same time. The cross-species separation rule (§V) compensates by repelling different species harder than same-species crowding, so that schools remain recognizable around the food source.

D. Status Indicators

A small status dot is rendered above each fish, colored red when hunger exceeds 0.6 and green otherwise. The dots are deliberately implemented with `meshBasicMaterial` so they are unaffected by the underwater lighting; they are intended as a perceptual debugging channel for the report writer and the demo audience, not as part of the simulated world.

VII. ENVIRONMENT, LANDMARKS, AND THE JELLYFISH HAZARD

A. Static Landmarks and Collision Footprints

The simulation volume is dressed with Bikini Bottom landmarks: SpongeBob’s pineapple house, Patrick’s rock house, Squidward’s tiki head, the Krusty Krab, and a clustered downtown skyline. Each landmark contributes a vertical cylinder ($\mathbf{p}, r, h$) to the static obstacle set. The downtown cluster was originally loaded as one large `.glb`; we split it into 13 individually-positioned parts so that each tower carries a tuned radius rather than one oversized circumscribed footprint that would block paths between buildings.

Obstacle constraints are enforced in two stages: a proactive lateral steer at force-computation time (§V) and a final position-snap-out step that runs twice per tick to handle the case in which two adjacent buildings would together pin an agent.

B. Jellyfish Swarm

The jellyfish hazard is modeled as a small swarm of `HAZARD_SWARM_SIZE = 10` members. Member 0 is the *leader*; it carries an explicit target position and a steering controller that mixes a target-seek vector with a slow time-varying “current” sine field, then clamps by a leader-specific maximum speed and force. The remaining nine members are *followers*. Each follower stores an offset around the leader; per tick its target position is the leader’s position plus the offset rotated by a slowly drifting yaw, plus a small vertical sine bob. A spring-damper-like force pulls the follower to that target with extra coupling to the leader’s velocity, producing a swarm that swims as one cohesive body but visibly shifts shape over time.

C. Periodic Bait Visits

On a randomized 30–45 s interval the leader replaces its exploration target with a point slightly above the Krabby Patty (`HAZARD_BAIT_HEIGHT_OFFSET`). Once the leader reaches the bait area the interval is re-randomized and the leader returns to random exploration. This creates an intermittent predator/bait dynamic: fish are attracted to the Krabby Patty, but the *same* location is periodically dangerous, and feeding fish must scatter when the swarm arrives. Figure 4 captures the moment when the swarm migrates over the bait area and the feeding fish disperse.

D. Procedural Jellyfish Animation

Rather than play a baked GLB animation, each jellyfish is rendered as three procedurally animated parts: a bell that scales in/out to simulate pulse, a body that bobs vertically out of phase with the bell, and four ribbon-like tentacles that wave with offset phases. This gives a more aquatic look than a fixed clip and allows every swarm member to use a slightly different phase offset so that the swarm does not pulse in lockstep.

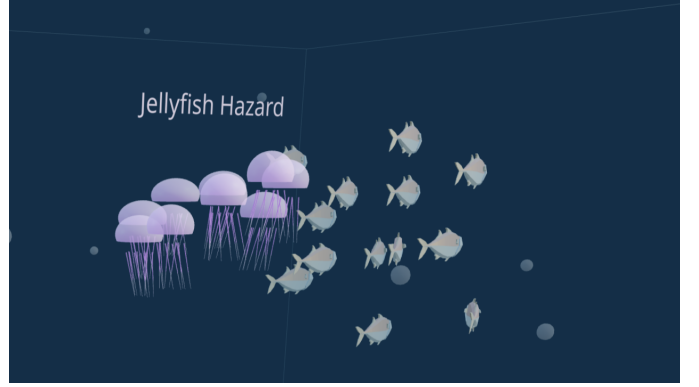


Fig. 4. Hazard avoidance during a periodic bait visit. The jellyfish swarm migrates over the Krabby Patty roughly every 30–45 s; feeding fish that fall inside the alert range generate a coordinated avoidance burst.

VIII. IMPLEMENTATION

A. Stack

The simulation is implemented in TypeScript and runs entirely in the browser. Three.js [7] provides the WebGL renderer and scene graph; React Three Fiber [8] exposes the scene graph as React components so that simulation state and UI state share the same reactive lifecycle; `@react-three/drei` [9] supplies `OrbitControls`, `Environment`, `Float`, `Text`, and animation helpers; Vite [10] runs the development server and produces the production bundle.

B. Code Layout

The repository is small and readable. The simulation logic is pure TypeScript with no React dependencies:

- `src/sim/types.ts` — shared data types.
- `src/sim/simulation.ts` — agent initialization, hunger update, steering forces, school-target modes, and the jellyfish swarm update.
- `src/sim/staticEnvironment.ts` — landmark placement and the static collision-footprint list.

The rendering and UI live alongside it:

- `src/components/SimulationScene.tsx` — Three.js scene, animation loop, status indicators, WASD camera.
- `src/components/ControlsPanel.tsx` — interactive sliders bound to the live config object.

C. Simulation Tick versus Render Tick

The simulation state is held in a ref and advanced by `stepSimulation` on every `useFrame` callback. The delta-time is clamped at 40 ms to keep the explicit integrator stable when the browser tab is backgrounded. Because state mutates the ref directly rather than going through React’s reconciler, no component re-renders per frame; React Three Fiber only updates the underlying Three.js objects.

D. GLB Model Reuse

Loaded .glb models are reused across many agents. Static landmarks use drei's lightweight `<Clone>`, but animated fish require an independent skeleton per instance; we use `SkeletonUtils.clone` from the Three.js examples to give each fish its own `AnimationMixer`.

E. Camera Navigation

The default camera uses drei's `OrbitControls` for mouse orbit and zoom. We add WASD translation on top by projecting the camera's forward vector onto the horizontal plane to get a stable heading, deriving the right vector from that heading, and translating both `camera.position` and `controls.target` by the same offset. Translating both keeps the orbit pivot in front of the camera so subsequent orbit input remains intuitive after a long WASD walk.

F. Interactive Controls

The control panel exposes `Agents`, `Perception radius`, `Food attraction`, `Hazard avoidance`, `Separation`, `Alignment`, `Cohesion`, a `Status dots toggle`, and a `Reset simulation` button. All sliders update the live `SimulationConfig`; the `Agents` slider additionally triggers `resizePopulation` to add or remove agents in-place rather than restarting the simulation.

IX. RESULTS AND OBSERVATIONS

A. Qualitative Behaviors

We report the behaviors visible at the default parameters (Table I).

a) Schooling and species separation.: Each of the four fish species starts in a different region of the simulation volume. Because alignment and cohesion are restricted to same-species neighbors and cross-species separation is widened, the four schools remain visually distinguishable even after they collectively visit the food source.

b) Hunger-driven feeding cycles.: Hungry fish (red status dots) accelerate toward the Krabby Patty when it enters their alert range. Once near the food, their hunger drops at 0.16/s and their dots turn green within roughly 6 seconds. Away from food, hunger recovers at 0.034/s, so a full fish returns to the hungry state after roughly 30 seconds of exploration — producing visible feeding cycles without any hard-coded timer.

c) School-level target modes.: With the default food-attraction weight (1.15, below the food-loiter threshold of 2.0), full schools enter the *avoid-food* mode and visibly drift off the Krabby Patty after feeding. Raising the slider past the threshold flips full schools into the *food-loiter* mode and they instead orbit the food source between feeding bursts.

d) Predator/bait dynamic.: Roughly every 30–45 seconds the jellyfish leader sets a target above the Krabby Patty and the swarm visibly migrates to the bait area. Feeding fish then scatter into a hazard-avoidance burst and re-converge once the swarm departs.

e) Obstacle navigation.: Ground-walking characters and fish near the seafloor steer laterally around the pineapple, the rock house, the Krusty Krab, and the downtown towers. The lateral steer plus the per-frame position snap-out gives a noticeable but smooth “slipping past” behavior rather than a head-on stall.

B. Parameter Sensitivity

The Food attraction slider is the most informative control. Sweeping it from 0.0 to 2.5 at default population produces three qualitative regimes:

- ≤ 0.4 : fish ignore the Krabby Patty; hunger climbs and feeding cycles fail to start.
- 0.4–2.0: visible feeding cycles; full schools drift away after eating.
- ≥ 2.0 : full schools enter loiter mode and remain clustered around the food source between feeding bursts.

C. Demo

A recorded live demo video is available at https://drive.google.com/file/d/1-6HV3sMmmmtutmhrwmW3iKqfHTpm_UdiU/view?usp=sharing, and the full source code is available at https://github.com/byte888/CS275_Final_Project.

X. LIMITATIONS AND FUTURE WORK

A. Quantitative Metrics

The current evaluation is qualitative. Future work could log average same-school neighbor distance, group cohesion, and hazard-avoidance reaction times, and use them to quantify the behavioral differences between parameter regimes.

B. Per-Species Hunger Dynamics

All fish currently share the same hunger drop and recovery rates. Giving each species its own parameters would produce more distinct ecological roles — for instance, slowly-feeding species could spend more time near the food and faster-recovering species could wander more.

C. Continuous Status Encoding

The hunger indicator is currently a binary red/green dot. A continuous color gradient or size encoding would convey hunger magnitude more precisely.

D. Smarter Predator Behavior

The jellyfish leader uses a random interval and a waypoint controller rather than explicit predator decision-making. A future version could let the leader bias its target toward whichever school is most densely clustered near the food, producing a more goal-directed predator.

E. Splitting the Food-Attraction Slider

The Food attraction slider controls both the magnitude of the steering force and whether a full school enters *food-loiter* or *avoid-food* mode. This is convenient for the demo but conflates two distinct phenomena. A more experimental version would split it into “food cue strength” and “food persistence.”

F. Tuning for Visual Clarity

The presentation’s Future Work slide notes that the behavior weights could be retuned to make schooling, clustering, and avoidance visually sharper. The interactive controls panel already supports this; a structured sweep with screenshots would make a useful addition to the report.

XI. CONCLUSION

We built a browser-based 3D artificial-life simulation in which every visible movement is explainable from local perception, an internal hunger state, and a school-level target mode. The system extends the classical boids backbone with two artificial-life-style mechanisms — per-agent hunger that gates food desire, and school-level target modes that stabilize group-scale wandering — and pairs them with cross-species separation, proactive lateral obstacle avoidance, and a procedurally animated jellyfish swarm that periodically visits the food source as a moving hazard. The result is a runnable, interactively tunable Bikini Bottom scene in which the same scaffold demonstrates schooling, feeding cycles, species separation, predator/bait dynamics, and landmark navigation.

APPENDIX

APPENDIX A: REPOSITORY AND DEMO

The full source code is available at https://github.com/byte888/CS275_Final_Project, and the recorded live demo video is available at https://drive.google.com/file/d/1-6HV3sMmmututmrwmW3iKqfHTpm_UdiU/view?usp=sharing.

APPENDIX B: ASSET SOURCES

3D models are taken from Poly Pizza [4], Quaternius [5], Kenney [6], and Sketchfab [11]. The jellyfish swarm is animated procedurally in JavaScript rather than from a baked clip.

REFERENCES

- [1] C. W. Reynolds, “Flocks, herds, and schools: A distributed behavioral model,” *ACM SIGGRAPH Computer Graphics*, vol. 21, no. 4, pp. 25–34, 1987.
- [2] V. Braitenberg, *Vehicles: Experiments in Synthetic Psychology*. MIT Press, 1984.
- [3] X. Tu, “Artificial animals for computer animation: Biomechanics, locomotion, perception, and behavior,” Ph.D. dissertation, Department of Computer Science, University of Toronto, 1996.
- [4] “Poly pizza — free low-poly 3d models,” <https://poly.pizza/>, asset source.
- [5] “Quaternius — free game-ready assets,” <https://quaternius.com/>, asset source.
- [6] “Kenney — free game assets,” <https://kenney.nl/assets>, asset source.
- [7] “Three.js,” <https://threejs.org/>.
- [8] “React three fiber,” <https://docs.pmnd.rs/react-three-fiber>.
- [9] “@react-three/drei,” <https://github.com/pmndrs/drei>.
- [10] “Vite,” <https://vitejs.dev/>.
- [11] “Sketchfab,” <https://sketchfab.com/>, asset source.